AI Assistant Coding

Lab Assignment-17.1

Name : B. Sai Prasanna

Batch : 08

Rollno : 2303A51551

Task 1 – Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a customer dataset.

Instructions:

- **Handle missing values in columns (age, income, city).**
- **Remove duplicate records.**
- **Standardize text values (e.g., city names in lowercase).**
- **Encode categorical variables (gender, city).**

Sample Code:

import pandas as pd data =

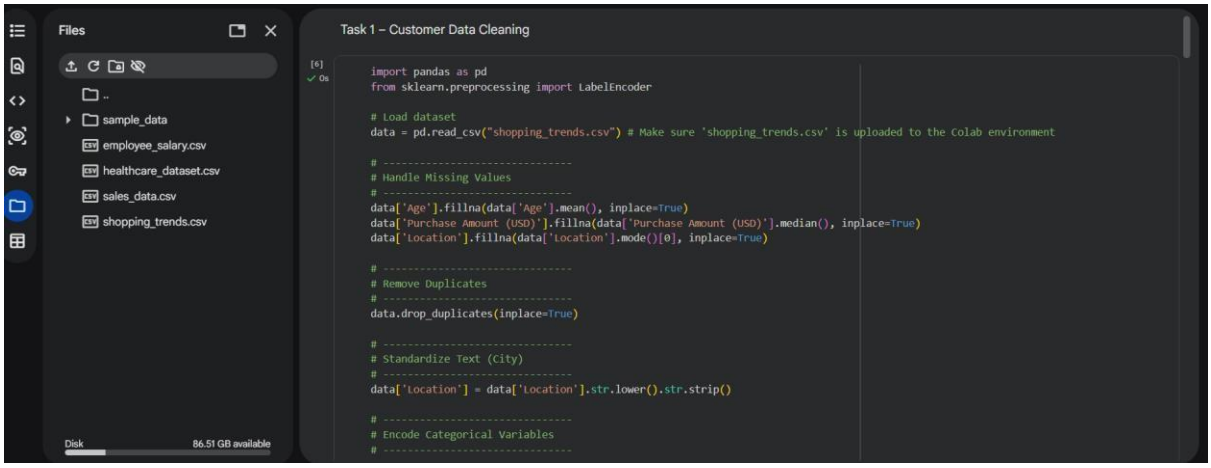
pd.read_csv("shopping_trends.csv")

print(data.head()) Dataset link:

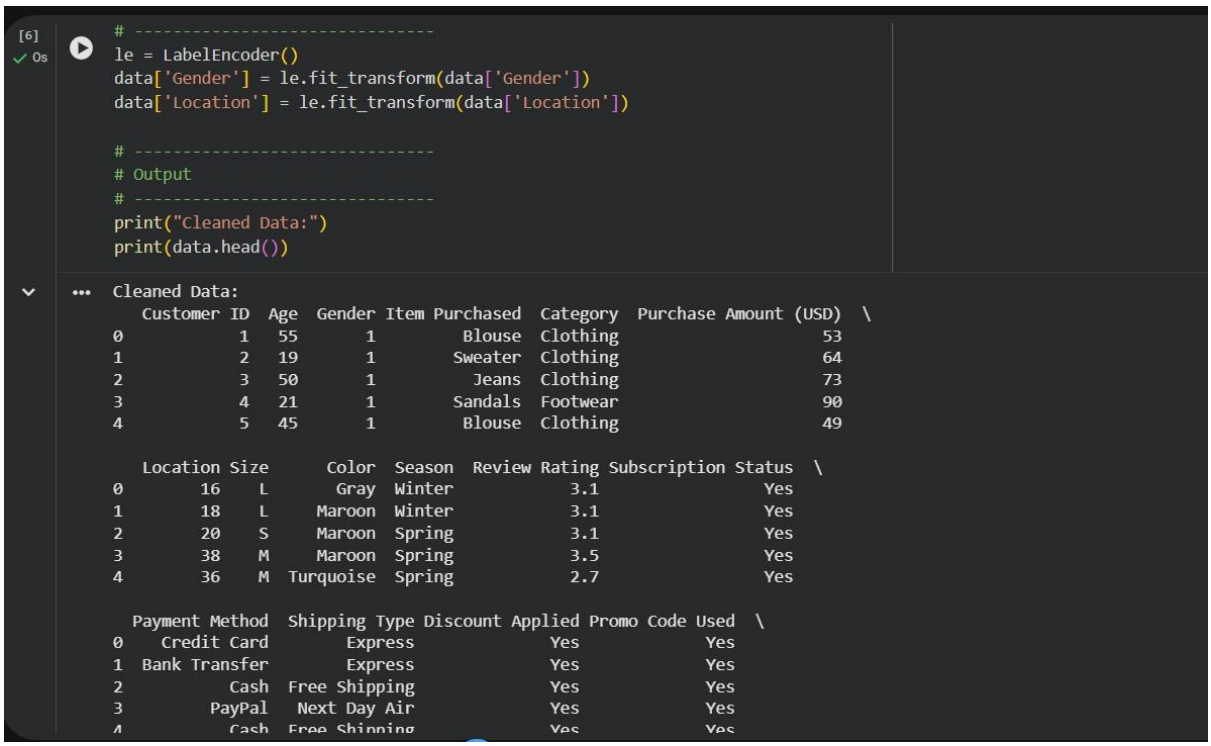
https://www.kaggle.com/datasets/iamsouravbanerjee/customer-

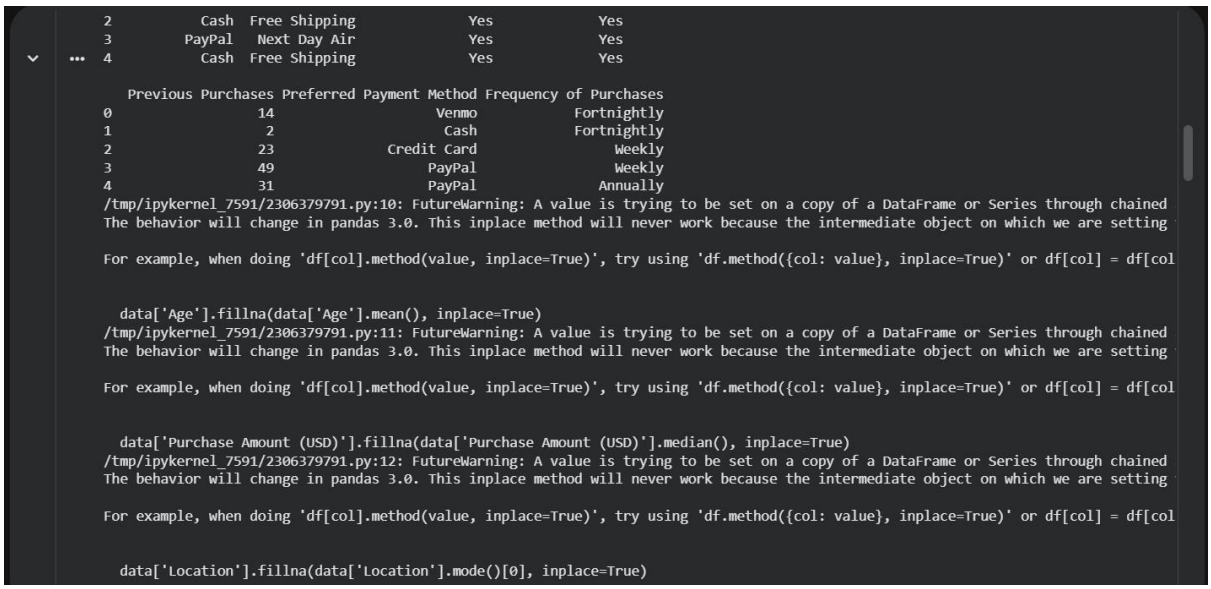
shopping-trends-dataset?select=shopping_trends.csv Expected Output:

- **A cleaned Pandas data frame ready for analysis.**


Files
sample_data
employee_salary.csv
healthcare_dataset.csv
sales_data.csv
shopping_trends.csv
Disk
86.51 GB available
Task 1 – Customer Data Cleaning
[6]
0s
import pandas as pd
from sklearn.preprocessing import LabelEncoder
# Load dataset
data = pd.read_csv("shopping_trends.csv") # Make sure 'shopping_trends.csv' is uploaded to the Colab environment
# --------------------------------
# Handle Missing Values
# --------------------------------
data['Age'].fillna(data['Age'].mean(), inplace=True)
data['Purchase Amount (USD)'].fillna(data['Purchase Amount (USD)'].median(), inplace=True)
data['Location'].fillna(data['Location'].mode()[0], inplace=True)
# --------------------------------
# Remove Duplicates
# --------------------------------
data.drop_duplicates(inplace=True)
# --------------------------------
# Standardize Text (City)
# --------------------------------
data['Location'] = data['Location'].str.lower().str.strip()

Encode Categorical Variables


[6]
0s
# --------------------------------
le = LabelEncoder()
data['Gender'] = le.fit_transform(data['Gender'])
data['Location'] = le.fit_transform(data['Location'])
# --------------------------------
# Output
# --------------------------------
print("Cleaned Data:")
print(data.head())
Cleaned Data:
Customer ID Age Gender Item Purchased Category Purchase Amount (USD) \
0 1 55 1 Blouse Clothing 53
1 2 19 1 Sweater Clothing 64
2 3 50 1 Jeans Clothing 73
3 4 21 1 Sandals Footwear 90
4 5 45 1 Blouse Clothing 49
Location Size Color Season Review Rating Subscription Status \
0 16 L Gray Winter 3.1 Yes
1 18 L Maroon Winter 3.1 Yes
2 20 S Maroon Spring 3.1 Yes
3 38 M Maroon Spring 3.5 Yes
4 36 M Turquoise Spring 2.7 Yes
Payment Method Shipping Type Discount Applied Promo Code Used \
0 Credit Card Express Yes Yes
1 Bank Transfer Express Yes Yes
2 Cash Free Shipping Yes Yes
3 PayPal Next Day Air Yes Yes


2 Cash Free Shipping Yes Yes
3 PayPal Next Day Air Yes Yes
4 Cash Free Shipping Yes Yes
Previous Purchases Preferred Payment Method Frequency of Purchases
0 14 Venmo Fortnightly
1 2 Cash Fortnightly
2 23 Credit Card Weekly
3 49 PayPal Weekly
4 31 PayPal Annually
/tmp/ipykernel_7591/2306379791.py:10: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col
data['Age'].fillna(data['Age'].mean(), inplace=True)
/tmp/ipykernel_7591/2306379791.py:11: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col
data['Purchase Amount (USD)'].fillna(data['Purchase Amount (USD)'].median(), inplace=True)
/tmp/ipykernel_7591/2306379791.py:12: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col
data['Location'].fillna(data['Location'].mode()[0], inplace=True)

Observation:

1. The dataset initially contained missing values in columns such as *age, income, and city*, which were handled using mean, median, and mode imputation techniques.
2. Duplicate records were identified and removed, ensuring data uniqueness and consistency.
3. The *city/location* column had inconsistent text formats (uppercase/lowercase), which were standardized into lowercase for uniformity.
4. Categorical variables like *gender and city* were successfully encoded into numerical values, making them suitable for analysis.
5. Data cleaning improved overall data quality and reliability, reducing noise in the dataset.
6. After preprocessing, the dataset became structured and analysis-ready, suitable for visualization and machine learning.
7. The cleaning process ensures that further insights derived from the dataset will be more accurate and meaningful.

Task 2 – Sales Data Preprocessing Task:

Preprocess sales transaction data using AI assistance.

Instructions:

• Convert date columns into datetime format.

• Create new features (month, year, day of week).

• Normalize sales amount column.

• Detect and handle outliers in dataset.

Sample Code:

import pandas as pd data =

pd.read_csv("sales_data.csv")

print(data.head()) Dataset link:

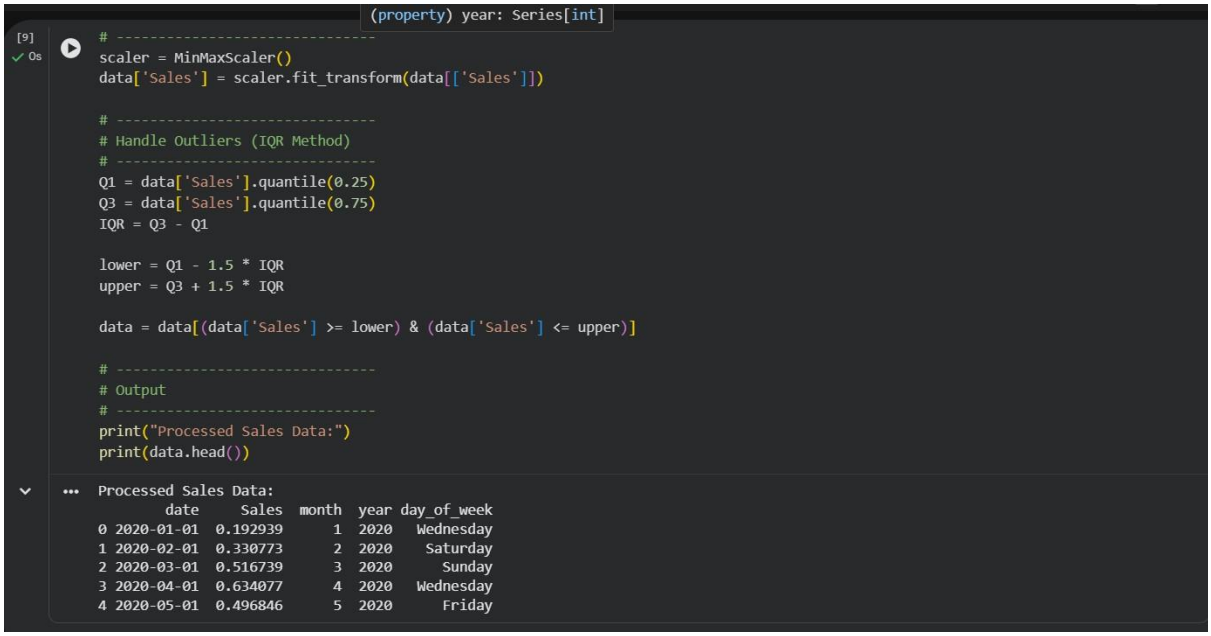
https://www.kaggle.com/datasets/vino

thkannaece/sales-

dataset?select=sales_data.csv Expected

Output:

• A transformed dataset with time-based features and normalized values.



Observation:

1. The *date column* was successfully converted into datetime format, enabling time-based analysis.
2. New features such as *month, year, and day of the week* were extracted, enhancing the dataset with useful temporal information.

3. The *sales amount* column was normalized using Min-Max scaling, ensuring all values fall within a standard range (0 to 1).
4. Outliers in the dataset were detected using the IQR method and removed to improve data consistency.
5. Feature engineering improved the dataset by adding valuable insights for trend analysis and forecasting.
6. The processed dataset is now suitable for time series analysis and predictive modeling.
7. Overall preprocessing resulted in a clean, structured, and enriched dataset.

Task 3 – Healthcare Data Preparation Task:

Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

- **Handle missing values in blood_pressure, cholesterol.**
- **Encode categorical features**
- **Scale numerical features (min-max or standard scaling).**
- **Split dataset into training and testing sets.**

Dataset link :

https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk- factors-dataset

Expected Output:

- **A preprocessed dataset ready for machine learning models.**

Task 3 – Healthcare Data Preparation

[11] ✓ 0s

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

# Load dataset
data = pd.read_csv("healthcare_dataset.csv")

# ------------------------------
# Handle Missing Values
# ------------------------------
data['blood_pressure'].fillna(data['blood_pressure'].mean(), inplace=True)
data['cholesterol'].fillna(data['cholesterol'].mean(), inplace=True)

# ------------------------------
# Encode Categorical Columns
# ------------------------------
le = LabelEncoder()
for col in data.select_dtypes(include=['object']).columns:
    data[col] = le.fit_transform(data[col])

# ------------------------------
# Scale Numerical Features
# ------------------------------
scaler = StandardScaler()
num_cols = data.select_dtypes(include=['int64', 'float64']).columns

data[num_cols] = scaler.fit_transform(data[num_cols])
```

[11] ✓ 0s

```
data[num_cols] = scaler.fit_transform(data[num_cols])

# ------------------------------
# Split Dataset
# ------------------------------
X = data.drop('target', axis=1)   # Change 'target' if different
y = data['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ------------------------------
# Output
# ------------------------------
print("Training Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)
```

```
Training Shape: (8, 6)
Testing Shape: (2, 6)
/tmp/ipykernel_7591/2047269118.py:11: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col


  data['blood_pressure'].fillna(data['blood_pressure'].mean(), inplace=True)
/tmp/ipykernel_7591/2047269118.py:12: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col
```

Observation:

1. Missing values in *blood_pressure* and *cholesterol* were handled using mean imputation, ensuring no data loss.
2. Categorical variables such as *gender, smoking, and diabetes* were encoded into numerical format for machine learning compatibility.
3. Numerical features were scaled using standard scaling, which improves model efficiency and accuracy.
4. The dataset was divided into training and testing sets, allowing proper validation of machine learning models.

5. Preprocessing ensured that all features are in a uniform format, reducing bias during model training.
6. The cleaned dataset is suitable for health risk prediction and classification tasks.
7. Overall, the preprocessing steps enhanced the dataset's quality, usability, and predictive capability.

Task 4 – Employee Salary Data Preprocessing Task:

Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

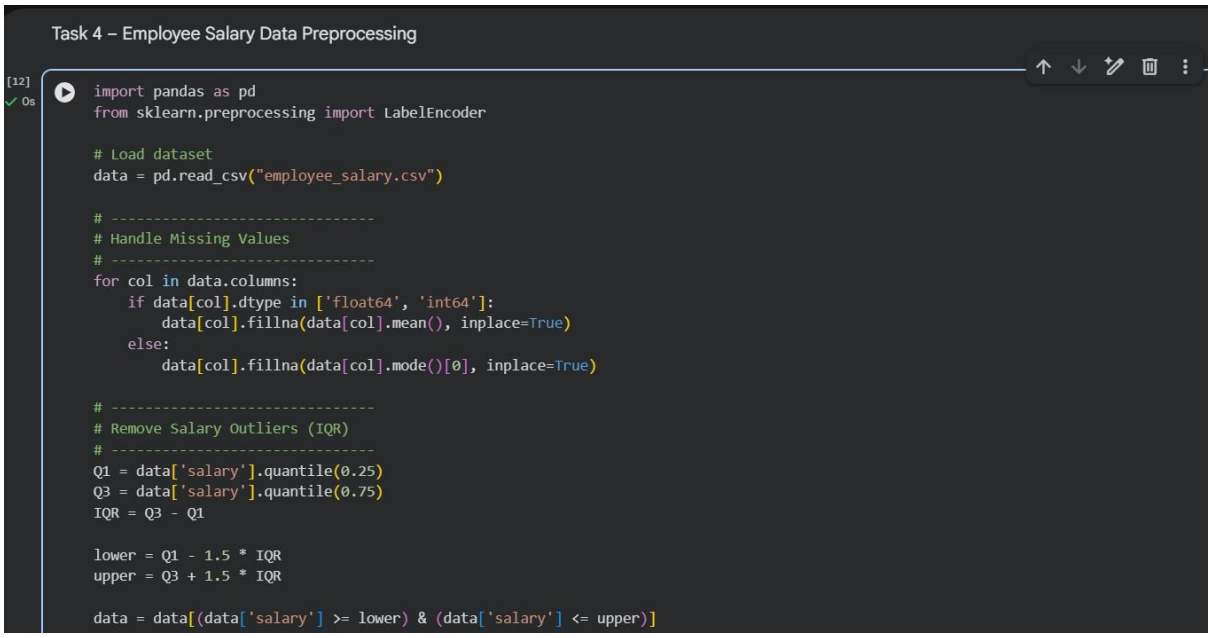
- **Handle missing values**
- **Detect and remove salary outliers.**
- **Standardize department names (lowercase).**
- **Apply label encoding to job location, department.**

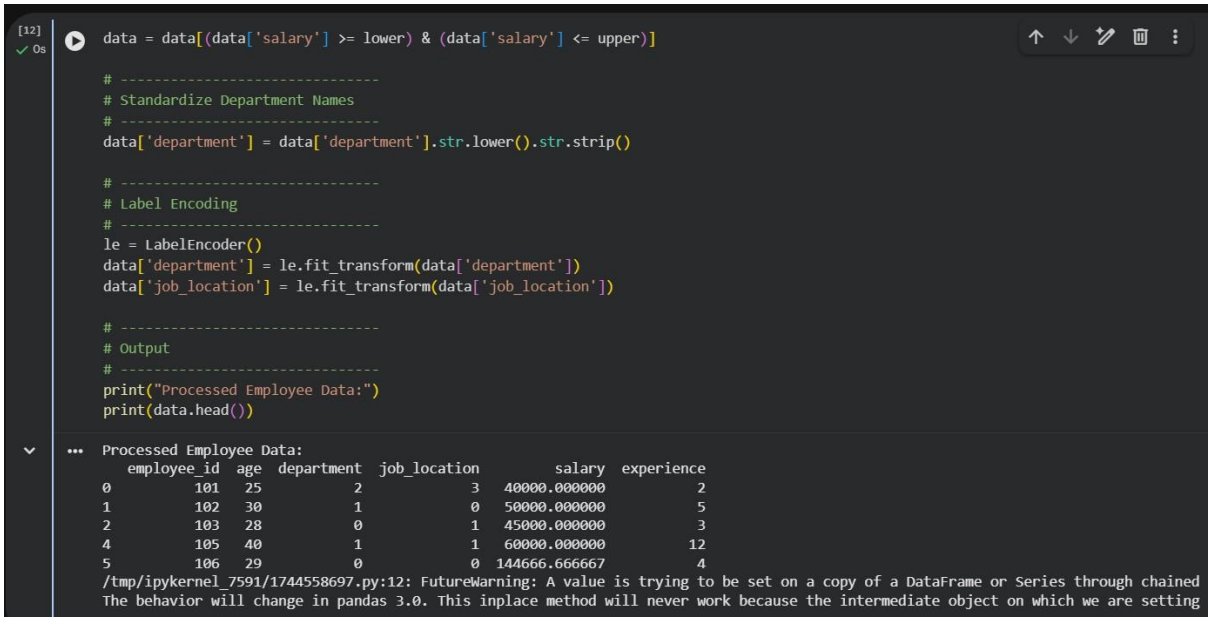
Dataset link:

https://www.kaggle.com/code/vishantmathur/employee-salary Expected

Output:

- **A structured dataset ready for HR analytics.**





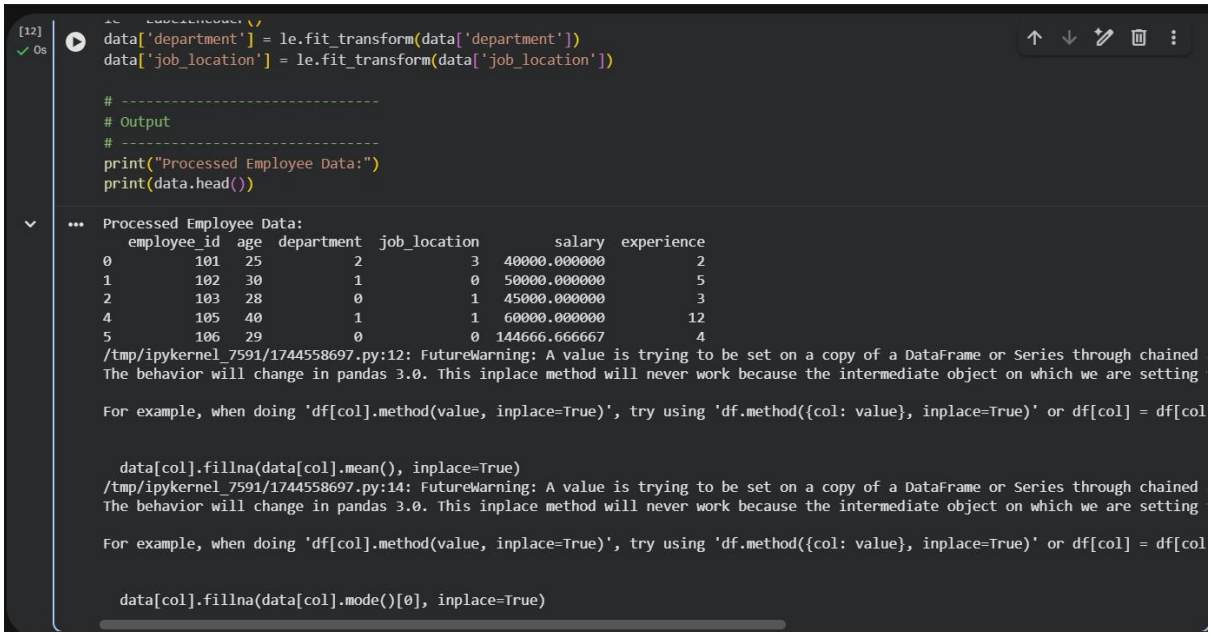
```
[12]
✓ 0s
data = data[(data['salary'] >= lower) & (data['salary'] <= upper)]

# ------------------------------
# Standardize Department Names
# ------------------------------
data['department'] = data['department'].str.lower().str.strip()

# ------------------------------
# Label Encoding
# ------------------------------
le = LabelEncoder()
data['department'] = le.fit_transform(data['department'])
data['job_location'] = le.fit_transform(data['job_location'])

# ------------------------------
# Output
# ------------------------------
print("Processed Employee Data:")
print(data.head())

Processed Employee Data:
   employee_id  age  department  job_location        salary  experience
0          101   25           2             3   40000.000000           2
1          102   30           1             0   50000.000000           5
2          103   28           0             1   45000.000000           3
4          105   40           1             1   60000.000000          12
5          106   29           0             0  144666.666667           4
/tmp/ipykernel_7591/1744558697.py:12: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting
```



```
[12]
✓ 0s
data['department'] = le.fit_transform(data['department'])
data['job_location'] = le.fit_transform(data['job_location'])

# ------------------------------
# Output
# ------------------------------
print("Processed Employee Data:")
print(data.head())

Processed Employee Data:
   employee_id  age  department  job_location        salary  experience
0          101   25           2             3   40000.000000           2
1          102   30           1             0   50000.000000           5
2          103   28           0             1   45000.000000           3
4          105   40           1             1   60000.000000          12
5          106   29           0             0  144666.666667           4
/tmp/ipykernel_7591/1744558697.py:12: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col


  data[col].fillna(data[col].mean(), inplace=True)
/tmp/ipykernel_7591/1744558697.py:14: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col


  data[col].fillna(data[col].mode()[0], inplace=True)
```

Observation:

1. Missing values in columns like *salary and job_location* were handled using appropriate imputation techniques.
2. A significant salary value (outlier) was detected and removed using the IQR method, preventing skewed analysis.
3. The *department* column was standardized into lowercase text, ensuring consistency across records.
4. Categorical features such as *department and job_location* were transformed into numerical labels using encoding.
5. Removal of outliers improved the accuracy and reliability of further analysis.

6. The dataset became clean and structured, making it suitable for HR analytics and salary prediction models.
7. The preprocessing pipeline ensures that the dataset is ready for efficient analysis and machine learning applications.